MCP

https://modelcontextprotocol.io/docs/getting-started/intro
https://modelcontextprotocol.io/docs/learn/architecture
https://modelcontextprotocol.io/examples

LangChain MCP Adapters - https://docs.langchain.com/oss/python/langchain/mcp

MCP Docs - https://modelcontextprotocol.io/docs
MCP Servers - https://github.com/modelcontextprotocol/servers

GitHub MCP Server

```
from dotenv import load_dotenv
load_dotenv()
import asyncio
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent
import os

GITHUB_TOKEN = os.getenv("GITHUB_TOKEN")
async def run_agent():
  client = MultiServerMCPClient(
      {
          "github": {
              "command": "npx",
              "args": [
                  "-y",
                  "@modelcontextprotocol/server-github"
              ],
              "env": {
                  "GITHUB_PERSONAL_ACCESS_TOKEN": GITHUB_TOKEN
              },
              "transport": "stdio"
          }
      }
  )
  tools = await client.get_tools()
  agent = create_agent("openai:gpt-4o", tools)
  response = await agent.ainvoke({"messages": "what are the files present in repository
    keertipurswani/EducosysGenerativeAI"})
  print(response["messages"][-1].content)

if __name__ == "__main__":
  asyncio.run(run_agent())
```

Filesystem MCP Server

https://github.com/modelcontextprotocol/servers/tree/main/src/filesystem

```
from dotenv import load_dotenv
load_dotenv()
import asyncio
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent
```

```
import os

GITHUB_TOKEN = os.getenv("GITHUB_TOKEN")
async def run_agent():
  client = MultiServerMCPClient(
      {
          "github": {
              "command": "npx",
              "args": [
                  "-y",
                  "@modelcontextprotocol/server-github"
              ],
              "env": {
                  "GITHUB_PERSONAL_ACCESS_TOKEN": GITHUB_TOKEN
              },
              "transport": "stdio"
          },
          "filesystem": {
              "command": "npx",
              "args": [
                  "-y",
                  "@modelcontextprotocol/server-filesystem",
                  "Todo… path"
              ],
              "transport":"stdio"
          }
      }
  )

  tools = await client.get_tools()
  agent = create_agent("openai:gpt-4o-mini", tools)
  response = await agent.ainvoke({"messages": "what are the files present in
    /Users/keertipurswani/GitHub/genai ref Directory"})
  print(response["messages"][-1].content)

if __name__ == "__main__":
  asyncio.run(run_agent())
```

```
   response = await agent.ainvoke({"messages": "Create a new file 'main6.py' in GenAI
    Directory"})
```

Browser Automation - Playwright

https://github.com/microsoft/playwright-mcp

```
from dotenv import load_dotenv
load_dotenv()
from langchain.agents.middleware import ModelCallLimitMiddleware
from langchain.agents.middleware import ToolCallLimitMiddleware


import asyncio
import os
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

async def run_agent():
   client = MultiServerMCPClient(
       {
           "playwright": {
               "command": "npx",
               "args": [
                   "-y",
                   "@playwright/mcp@latest"
               ],
               "transport": "stdio"
```

```
            }
        }
    )

    tools = await client.get_tools()
    agent = create_agent(
        "openai:gpt-5.5",
        tools,
        middleware=[
                ModelCallLimitMiddleware(run_limit=15, exit_behavior="end"),
                ToolCallLimitMiddleware(tool_name="browser_navigate", run_limit=5,
    exit_behavior="end"),
                ToolCallLimitMiddleware(tool_name="browser_snapshot", run_limit=5,
    exit_behavior="end"),
                ToolCallLimitMiddleware(tool_name="browser_click", run_limit=5,
    exit_behavior="end"),
            ]
        )

    response = await agent.ainvoke({"messages": "Go to https://google.com, search for 'Model
    Context Protocol', and summarize the top result"})
    print(response["messages"][-1].content)

if __name__ == "__main__":
    asyncio.run(run_agent())
```

Custom Server

https://reference.langchain.com/python/langchain-mcp-adapters

Terminal Command Runner

mcp_demo.py

MCP client that connects to server, attaches its tools to agent, and sends it a task to execute.

```
from dotenv import load_dotenv
load_dotenv()
import asyncio
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent
import os


async def run_agent():
 client = MultiServerMCPClient(
     {
         "TerminalCmdRunner": {
           "command": "python",
           "args": [
               "./terminal_cmd_runner.py"
           ],
           "transport": "stdio"
       }
     }
 )

 tools = await client.get_tools()
 agent = create_agent("openai:gpt-4o", tools)
 response = await agent.ainvoke({"messages": "list all files in the present directory."})
 print(response["messages"][-1].content)

if __name__ == "__main__":
 asyncio.run(run_agent())
```

terminal_cmd_runner_mcp.py -

MCP server that exposes 2 tools to run shell commands, either from a default location or a specific directory.

```
# terminal_cmd_runner_mcp.py
from mcp.server.fastmcp import FastMCP
import subprocess

mcp = FastMCP("TerminalCmdRunner")

@mcp.tool()
def run_command(command: str) -> str:
   """Run a shell command and return its output"""
   result = subprocess.run(
       command,
       shell=True,
       capture_output=True,
       text=True
   )
   output = result.stdout
```

```
    if result.stderr:
        output += f"\nSTDERR: {result.stderr}"
    return output or "(no output)"

@mcp.tool()
def run_in_directory(command: str, directory: str) -> str:
    """Run a shell command inside a specific directory"""
    result = subprocess.run(
        command,
        shell=True,
        capture_output=True,
        text=True,
        cwd=directory
    )
    output = result.stdout
    if result.stderr:
        output += f"\nSTDERR: {result.stderr}"
    return output or "(no output)"


if __name__ == "__main__":
    mcp.run(transport="stdio")
```

FileSystem Server

```
from mcp.server.fastmcp import FastMCP
import os

mcp = FastMCP("EducosysFileSystem")

@mcp.tool()
def addFile(filename: str):
  """Create a new file in current directory"""
  if not os.path.exists(filename):
      with open(filename, "w") as f:
          pass
      print(f"File '{filename}' created.")
  else:
      print(f"File '{filename}' already exists.")


@mcp.tool()
def addFolder(directory_name: str):
  """Create a new Directory in current directory"""
  if not os.path.exists(directory_name):
      os.mkdir(directory_name)
      print(f"Directory '{directory_name}' created.")
  else:
      print(f"Directory '{directory_name}' already exists.")


if __name__ == "__main__":
  mcp.run(transport="stdio")
```

```
"EducosysFileSystem": {
                "command": "python",
                "args": [
                    "./filesystem_mcp.py"
                ],
                "transport":"stdio"
            }
```